পর্ব ১ · কনসেপ্ট · ক্লাস ১ / ৩৭

শূন্য থেকে শুরু: কী বানাচ্ছি আর কার জন্য

সিলেবাস: প্র্যাকটিক্যাল ১ · টপিক ১.১ ও ১.২

প্রজেক্ট চিহ্নিত করা · ব্যবহারকারী চেনা · স্কোপ বোঝা

IMS — Institute Management System

শুরু করুন প্রশ্ন দিয়ে

আপনার স্কুলে আজকের উপস্থিতি কোথায় লেখা হচ্ছে?

হাত তুলে উত্তর নিন — ২–৩ জন

খাতায়? রেজিস্টারে? এক্সেলে?

আর মাসিক রিপোর্ট বানাতে কত সময় লাগে?

এই তিনটে উত্তর থেকেই আজকের প্রজেক্টের জন্ম — আর তিনটেই আসলে **একই সমস্যার তিনটি মুখ**।

শিক্ষকের নোট: উত্তর যাই আসুক, থামবেন না। কেউ "এক্সেলে করি" বললে জিজ্ঞেস করুন — "দুই শিক্ষক একই ফাইলে একসাথে লিখলে কী হয়?"

ধাপ ১

সমস্যাটা কী

খাতা-ভিত্তিক ব্যবস্থার তিনটি বাস্তব ক্ষতি

📕

তথ্য হারায়

খাতা ভিজলে বা হারালে এক বছরের উপস্থিতি ও নম্বর একসাথে মুছে যায়। কোনো কপি নেই।

⏳

রিপোর্টে সময়

"গত মাসে কোন ক্লাসের উপস্থিতি সবচেয়ে খারাপ?" — এই এক প্রশ্নের উত্তর দিতে কয়েক ঘণ্টা হিসাব।

🕵️

দায় থাকে না

কে নম্বর বদলাল, কখন বদলাল — কোনো রেকর্ড নেই। ভুল হলে দায় কেউ নেয় না।

সফটওয়্যারের কাজ এক লাইনে: **তথ্য একবার লেখা হবে, অনেকবার কাজে লাগবে।**

শিক্ষকের নোট: বলুন — "তিনটি সমস্যার একটাও প্রযুক্তির নয়। তিনটিই তথ্য হারানো, সময় নষ্ট, আর জবাবদিহি। ভালো সফটওয়্যার শুরু হয় সমস্যা দেখে, প্রযুক্তি দেখে নয়।"

সিদ্ধান্ত ১

এক স্কুলের সফটওয়্যার নয় — একটি প্ল্যাটফর্ম

প্রজেক্টের আকার ঠিক করে দেওয়া প্রথম সিদ্ধান্ত

এক স্কুলের জন্য বানালে

২০টি স্কুলে দিতে হলে ২০টি কপি →
২০টি ডেটাবেস · ২০টি আপডেট · ২০টি ব্যাকআপ · ২০টি আলাদা বাগ

একজন ডেভেলপার সামলাতে পারবে না। প্রথম গ্রাহক এলেই থেমে যাবে।

এক প্ল্যাটফর্ম বানালে

একই সিস্টেমে অনেক প্রতিষ্ঠান →
একটি ডেটাবেস · একটি আপডেট · একটি ব্যাকআপ

প্রতিষ্ঠান নিজেই সাইনআপ করে, ১৪ দিন ট্রায়াল নেয়, তারপর সাবস্ক্রিপশন।

এই এক সিদ্ধান্তই ঠিক করে দিল ডেটাবেসের নকশা, লগইন, অনুমতি, বিলিং — সব। প্রজেক্টের প্রথম দিনে এটা ভাবা সবচেয়ে জরুরি কাজ।

শিক্ষকের নোট: বোর্ডে লিখুন — "এক স্কুল = কপি। অনেক স্কুল = প্ল্যাটফর্ম।" তারপর জিজ্ঞেস করুন: "আমাদের প্রজেক্ট কোনটা?"

ধারণা

একটি পণ্য, তিনটি স্তর

প্রতিটির ব্যবহারকারী আলাদা — তাই উন্নয়নেও আলাদা

পাবলিক সাইট

দাম দেখানো → সাইনআপ → ১৪ দিনের ফ্রি ট্রায়াল → লগইন

লগইনের আগে

কাজের সফটওয়্যার

উপস্থিতি · পাঠ · হোমওয়ার্ক · পরীক্ষা · রিপোর্ট কার্ড

লগইনের পরে

অপারেটর প্যানেল — /platform/

গ্রাহক প্রতিষ্ঠান · প্ল্যান · ট্রায়াল · ইনভয়েস

শুধু আপনার

শিক্ষকের নোট: তিনটে পর্দা লাইভ দেখান — প্রথমে pricing পেজ, তারপর লগইন করে ড্যাশবোর্ড, শেষে `/platform/`। বলুন: "একই কোড, তিনটে আলাদা পণ্য।"

ধারণা

পাঁচটি ভূমিকা

প্রতিটি ভূমিকা শুধু নিজের অংশটুকু দেখে — পর্দায় এবং সার্ভারে

অ্যা প্রতিষ্ঠান অ্যাডমিন

✓ ক্লাস-বিষয় তৈরি, শিক্ষক যোগ, ছাত্র ভর্তি, ফল প্রকাশ, ব্র্যান্ডিং, সাবস্ক্রিপশন নবায়ন

× অন্য প্রতিষ্ঠানের তথ্য

কো কোঅর্ডিনেটর

✓ একটি বিভাগ বা একটি ক্লাসের তদারকি — উপস্থিতি ও নম্বরের অবস্থা

× নিজের স্কোপের বাইরে যাওয়া, অ্যাডমিনের কাজ

শি শিক্ষক

✓ উপস্থিতি নেওয়া, পাঠ পরিকল্পনা, হোমওয়ার্ক দেওয়া, নম্বর তোলা

× ফল প্রকাশ, বিলিং, অন্য ক্লাসের নম্বর

ছা শিক্ষার্থী

✓ হোমওয়ার্ক জমা, নিজের ফল ও উপস্থিতি দেখা

× অন্য কারো ফল, নম্বর বদলানো

প্ল্যা প্ল্যাটফর্ম অ্যাডমিন — শুধু আপনার

✓ সব গ্রাহক প্রতিষ্ঠান, প্ল্যান, ট্রায়াল, ইনভয়েস — পুরো ব্যবসা

× কোনো প্রতিষ্ঠানের ভেতরের পড়াশোনার তথ্য

খেয়াল করুন — প্রতিটি ভূমিকার নিচে একটা লাল × আছে। **সেই লাল অংশটাই পরে আমরা কোডে আটকাব।**

শিক্ষকের নোট: পাঁচজনের দায়িত্ব পড়তে বলুন, তারপর জিজ্ঞেস করুন — "কে সবচেয়ে কম দেখবে?" উত্তর: শিক্ষার্থী। "কে সবচেয়ে বেশি?" উত্তর: অ্যাডমিন — কিন্তু সে-ও অন্য প্রতিষ্ঠান দেখে না।

কেন্দ্রীয় ধারণা

এক ডেটাবেস, আলাদা ঘর

প্রতিটি প্রতিষ্ঠান একটি Workspace — এর বাইরের কোনো আইডি দিলে ৪০৪

Workspace A — Sunrise Model School

- শিক্ষার্থী · উপস্থিতি · নম্বর
- ক্লাস · বিষয় · পরীক্ষা
- সাবস্ক্রিপশন · ইনভয়েস
- ব্র্যান্ডিং · লোগো

শিক্ষক, শিক্ষার্থী, কোঅর্ডিনেটর — সবাই এখানে

B-এর আইডি
A-তে খুঁজলে
৪০৪

Workspace B — Springfield Coaching

- শিক্ষার্থী · উপস্থিতি · নম্বর
- ক্লাস · বিষয় · পরীক্ষা
- সাবস্ক্রিপশন · ইনভয়েস
- ব্র্যান্ডিং · লোগো

সম্পূর্ণ আলাদা — একটাও সারি ভাগ করা নয়

"পাওয়া যায়নি" — "নিষেধ" নয়

শিক্ষকের নোট: ৪০৪ আর ৪০৩-এর পার্থক্যটা জিজ্ঞেস করুন। উত্তর: ৪০৩ বললে বোঝা যায় জিনিসটা আছে, শুধু অনুমতি নেই — সেটাও তথ্য ফাঁস। এই একটা পার্থক্যই এই প্রজেক্টের নিরাপত্তার ভিত্তি।

পার্থক্য

"ব্যবহারকারী" আর "গ্রাহক" এক নয়

এই পার্থক্য না বুঝলে বিলিং মডেল উল্টো হয়ে যাবে

	ব্যবহারকারী (user)	গ্রাহক (customer)
কে	শিক্ষক, শিক্ষার্থী, কোঅর্ডিনেটর, অ্যাডমিন	প্রতিষ্ঠান (স্কুল/কোচিং)
সংখ্যা	এক প্রতিষ্ঠানে ৫–৮০০ জন	এক প্রতিষ্ঠান = এক গ্রাহক
টাকা দেয়?	**না**	**হ্যাঁ** — সাবস্ক্রিপশন
অ্যাকাউন্ট বন্ধ হলে	নিজের লগইন হারায়, প্রতিষ্ঠান টিকে থাকে	প্রতিষ্ঠানের সেবা বন্ধ হয়
কোডে কোথায় বাঁধা	`User` + `Membership`	`Workspace` + `Subscription`

এই প্রজেক্টে একবার ভুল হয়েছিল: সাবস্ক্রিপশন বাঁধা ছিল ব্যক্তির সদস্যপদে। ফলে শিক্ষক প্রতিষ্ঠান ছাড়লে সাবস্ক্রিপশনও চলে যেত। ঠিক করা হয়েছে `Workspace`-এ বাঁধে।

শিক্ষকের নোট: ক্লাসে জিজ্ঞেস করুন — "তোমার প্রজেক্টে টাকা কে দেবে? ব্যবহারকারী, না প্রতিষ্ঠান?" উত্তরটা নিজের প্রজেক্টের ডেটা মডেলে লিখে রাখতে বলুন।

লাইভ

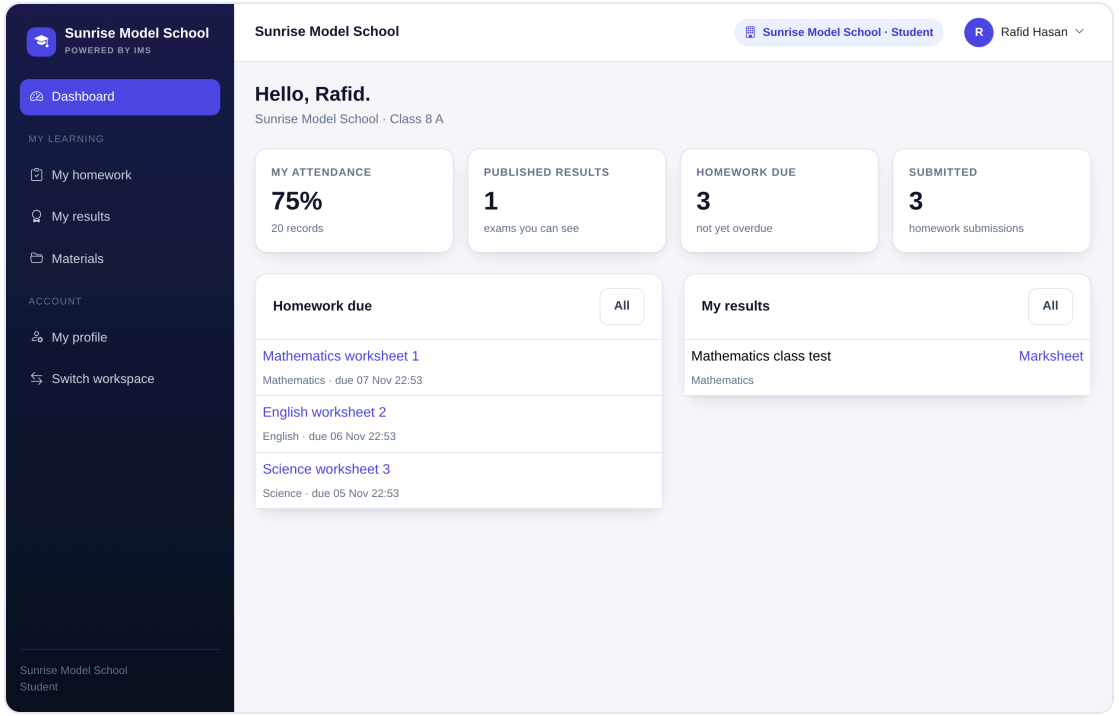
চারটি লগইন, চারটি ভিন্ন পর্দা

একই সিস্টেম — শুধু ভূমিকা বদলান, মেনু বদলে যায়

লগইন	ভূমিকা	মেনু সংখ্যা
`admin@ims.local`	প্রতিষ্ঠান অ্যাডমিন	সব
`coordinator@ims.local`	কোঅর্ডিনেটর	তদারকি
`teacher@ims.local`	শিক্ষক	৫টি
`student@ims.local`	শিক্ষার্থী	৩টি

```
cd ~/Projects/"Web Development Project"/ims
bash run.sh
→ http://127.0.0.1:8000/

সবার পাসওয়ার্ড:  DemoPass!2026
```



শিক্ষার্থীর পর্দা — মাত্র তিনটি মেনু

ক্লাসে করণীয়: অ্যাডমিন হয়ে মেনু **গুনুন** ও খাতায় লিখুন। তারপর শিক্ষার্থী হয়ে ঢুকে আবার গুনুন।

শিক্ষকের নোট: ছাত্ররাই বলবে কী কী হারিয়ে গেছে। এই মুহূর্তটাই রোলভিত্তিক ইন্টারফেস শেখানোর সবচেয়ে ভালো সুযোগ।

ক্লাসের সেরা মুহূর্ত

মেনুতে নেই, তবু ঢোকা যায়?

শিক্ষক হয়ে হাতে এই ঠিকানা লিখুন

```
/w/sunrise-model-school/billing/invoices/
```

ভুল ধারণা

"মেনুতে লিংক নেই, তাই কেউ ঢুকতে পারবে না।" — এটা নিরাপত্তা নয়, শুধু লুকানো।

আসল নিয়ম

মেনু লুকানো = **সুবিধা**। আটকাতে হয় **সার্ভারে** — তাই ৪০৩ পর্দা আসে।

নিয়ম: **একই তালিকা** ব্যবহার হয় মেনু আঁকতে এবং অনুমতি যাচাই করতে — `apps/core/navigation.py`।

শিক্ষকের নোট: "এই একটু জায়গাতেই বেশিরভাগ ছাত্রদের প্রজেক্টে ফাঁক থাকে — মেনু লুকিয়ে দেয়, কিন্তু URL দিয়ে ঢোকা আটকায় না। পরীক্ষায় এটা জিজ্ঞেস করা হবে।"

প্রমাণ

"কাজ করছে" বলার আগে স্ক্রিপ্টে দেখান

৫৫ পেজ × ৪ ভূমিকা = ২২০টি স্বয়ংক্রিয় যাচাই

```
python smoke_test.py --base http://127.0.0.1:8000

  ORG_ADMIN     55/55 as expected
  COORDINATOR   55/55 as expected
  TEACHER       55/55 as expected
  STUDENT       55/55 as expected

PASS: 0 mismatched expectations
```

১০৩

স্বয়ংক্রিয় টেস্ট — `apps/*/tests.py`

২২০

ভূমিকা × পেজ যাচাই — `smoke_test.py`

১৯

Git commit — প্রতিটি সিদ্ধান্তের প্রমাণ

শিক্ষকের নোট: বলুন — "কেউ ভুল করে শিক্ষার্থীর জন্য বিলিং পেজ খুলে দিলে, এই স্ক্রিপ্ট পরের বার চালানোর সময়েই ধরে ফেলবে। আপনার না দেখেও।"

আলোচনা

ক্লাসে জিজ্ঞেস করুন

এই চারটি প্রশ্নের উত্তর নিজের প্রজেক্টে খুঁজতে হবে

- তোমার সিস্টেমে **কে কে** ব্যবহার করবে — তিনটি ভূমিকা লিখে ফেলো।
- প্রতিটি ভূমিকা **কোন জিনিসটা দেখতে পাবে না**? (এটাই আসল ডিজাইন)
- "গ্রাহক" আর "ব্যবহারকারী" কি তোমার প্রজেক্টে **একই মানুষ**?
- তোমার সিস্টেম **কী করবে না** — একটা উত্তর এখনই বলো।

শিক্ষকের নোট: শেষ প্রশ্নটা কঠিন — ২০ সেকেন্ড সময় দিন। যে উত্তর দিতে পারে না, তার স্কোপ এখনো ঠিক হয়নি। এটাই পরের ক্লাসের প্রবেশদ্বার।

সারাংশ

আজকের পাঁচটি শিক্ষা

ক্লাস শেষে বোর্ডে এগুলোই লিখে রাখুন

#	শিক্ষা	প্রমাণ
১	সমস্যা আগে, প্রযুক্তি পরে	তিনটি ক্ষতি — একটাও প্রযুক্তিগত নয়
২	ভূমিকা ঠিক মানে পর্দা ঠিক	৪টি লগইন, ৪টি ভিন্ন মেনু
৩	"দেখায় না" মানে "আটকায় না" নয়	৪০৩ পর্দা — সার্ভারে যাচাই
৪	ব্যবহারকারী ≠ গ্রাহক	`User` বনাম `Workspace`
৫	দাবির প্রমাণ থাকতে হয়	`smoke_test.py` — ২২০ যাচাই

শিক্ষকের নোট: ক্লাসের শেষ দুই মিনিট: প্রত্যেকে একটা বাক্যে বলবে — "আজকের কোন শিক্ষাটা আমার প্রজেক্টে সবচেয়ে দরকার?"

১

পরের ক্লাস · ক্লাস ২

ধারণা অনেক আছে, কোনটা বাছব?

১২টি প্রজেক্ট · ৫টি মাপকাঠি · একটি স্কোর টেবিল

পর্ব ১ · কনসেপ্ট · ক্লাস ২ / ৩৭

কোন প্রজেক্ট, আর কোনটা বাদ দেব

সিলেবাস: প্র্যাকটিক্যাল ১ · টপিক ১.৩ (প্রজেক্ট নির্বাচন ও স্কোপ)

স্কোর কার্ড · স্কোপ ফ্রিজ · ফিচার ক্যাটালগ

IMS — ৫৭ ফিচার · ৬ ভূমিকা · ৩১ টেবিল

শুরু করুন প্রশ্ন দিয়ে

বারোটা আইডিয়া, একটা সেমিস্টার — কোনটা নেবেন?

হাত তুলে বলুন: আপনার পছন্দের প্রজেক্টটা কী, আর কেন?

সবাই বলে "যেটা ভালো লাগে"।
কিন্তু ভালো লাগা দিয়ে প্রজেক্ট শেষ হয় না।

সেমিস্টারের শেষে দুই ধরনের প্রজেক্ট দেখা যায় — **একটা ডেমো দিতে পারে**, আরেকটা "প্রায় হয়ে গেছে" বলে চলে যায়।
পার্থক্যটা কোডিংয়ে তৈরি হয় না। তৈরি হয় **প্রথম সপ্তাহের সিদ্ধান্তে**।

শিক্ষকের নোট: কেউ "আমি ই-কমার্স বানাব" বললে থামিয়ে জিজ্ঞেস করুন — "কার ই-কমার্স? ক্রেতা কে, বিক্রেতা কে, টাকা কে নেবে?" উত্তর যত দেরিতে আসে, প্রজেক্টটা তত বেশি ঝুঁকিতে।

ধাপ ১

পাঁচটি মাপকাঠি — ১০ নম্বরের স্কোর কার্ড

প্রতিটি মাপকাঠিতে ০ / ১ / ২ — মোট সর্বোচ্চ ১০

👤

১. ব্যবহারকারী আছে?

২ = চেনা মানুষ প্রতিদিন ব্যবহার করবে
১ = কল্পনা করতে হচ্ছে
০ = "সবাই ব্যবহার করবে"

🧩

২. ডেটার জটিলতা

২ = সম্পর্ক আছে, ভূমিকা আছে
১ = শুধু তালিকা
০ = একটি টেবিলই যথেষ্ট

🔍

৩. প্রমাণ দেখানো যাবে?

২ = স্ক্রিনশট/টেস্ট দিয়ে দেখানো যায়
১ = আংশিক
০ = শুধু মুখে বলতে হবে

⏱️

৪. সময়ে শেষ হবে?

২ = ৮ সপ্তাহে MVP
১ = টানাটানি
০ = অসম্ভব

🔥

৫. নিজের আগ্রহ

২ = রাতে ঘুম আসবে না
১ = ঠিক আছে
০ = বাধ্য হয়ে

নিয়ম: **৩ নম্বর মাপকাঠিতে ০ পেলে বাকি স্কোর যতই হোক — প্রজেক্ট বাদ।** যে কাজের প্রমাণ দেখানো যায় না, সেটা কোর্সে "কাজ" হিসেবে গণ্য হয় না।

শিক্ষকের নোট: ৫ নম্বর (আগ্রহ) কে ছোট করে দেখবেন না — এটি একমাত্র মাপকাঠি যা ১২ সপ্তাহের ক্লান্তি সামলায়। কিন্তু এটি একা কখনোই যথেষ্ট নয়।

ধাপ ২

১২টি প্রজেক্টের স্কোর

ক্লাসের কাজ: নিজের পছন্দের প্রজেক্টটি এই টেবিলে বসিয়ে স্কোর দিন

#	প্রজেক্ট	ব্যবহারকারী	ডেটা	প্রমাণ	সময়	আগ্রহ	মোট /১০	সিদ্ধান্ত
১	**স্কুল/কলেজ ম্যানেজমেন্ট (IMS)**	২	২	২	১	২	**৯**	✓ নেওয়া হলো
২	রেস্টুরেন্ট POS ও বিলিং	২	২	১	২	২	**৯**	◐ টাই — পরে দেখুন
৩	হাসপাতাল অ্যাপয়েন্টমেন্ট	২	২	১	০	২	৭	○ বাদ
৪	গার্মেন্টস অ্যাটেনডেন্স	২	২	২	০	১	৭	○ বাদ
৫	লাইব্রেরি বই ইস্যু	১	১	২	২	১	৭	○ বাদ
৬	টিউশন বুকিং	২	১	১	১	১	৬	○ বাদ
৭	কৃষি পরামর্শ অ্যাপ	১	১	০	২	২	৬	✕ প্রমাণ ০
৮	রক্তদান ম্যাচিং	১	১	১	২	১	৬	○ বাদ
৯	ই-কমার্স (কাপড়)	২	১	১	০	১	৫	✕ সময় ০
১০	ফ্রিল্যান্স মার্কেটপ্লেস	১	২	১	০	১	৫	✕ সময় ০
১১	সোশ্যাল মিডিয়া ক্লোন	১	২	০	০	২	৫	✕ প্রমাণ ০
১২	ট্রাভেল বুকিং	১	২	১	০	০	৪	✕ সময় ০

খেয়াল করুন: **দুই নম্বরে ৯ আর এক নম্বরে ৯ — টাই।** স্কোর টেবিল একা সিদ্ধান্ত দেয় না, শুধু বাজে আইডিয়া ছেঁটে দেয়।

শিক্ষকের নোট: "সময় ০" মানে অসম্ভব নয় — মানে ওই প্রজেক্টের MVP এখনো কেউ সংজ্ঞায়িত করেনি। কেউ প্রতিবাদ করলে বলুন: "তাহলে বলো, প্রথম সপ্তাহে ঠিক কী কাজ করবে?"

ধাপ ৩

৯টি কেন বাদ পড়ল — তিনটি কারণে

বাদ দেওয়াও একটা সিদ্ধান্ত, লিখে রাখতে হয়

প্রমাণ দেওয়া যাবে না

কৃষি পরামর্শ, সোশ্যাল ক্লোন — ব্যবহারকারী পাওয়া যায় না, তাই স্ক্রিনশটের পেছনে গল্প ছাড়া কিছু নেই।

৩টি প্রজেক্ট

MVP সংজ্ঞায়িত হয়নি

ই-কমার্স, মার্কেটপ্লেস, ট্রাভেল — "পুরোটা" করতে গেলে ২ বছরের কাজ, "কোনটা বাদ" বলা যায়নি।

৪টি প্রজেক্ট

ডেটা খুব সরল

লাইব্রেরি, রক্তদান — শেখানোর জন্য ভালো, কিন্তু সম্পর্ক (relationship) শেখানোর মতো গভীরতা নেই।

২টি প্রজেক্ট

নিয়ম: **প্রতিটি বাদ দেওয়া আইডিয়ার পাশে এক লাইনে কারণ লিখুন।** তিন মাস পর "আমার আরেকটা আইডিয়া ছিল" বলে প্রজেক্ট বদলানো শুরু হয় ঠিক এই এক লাইন না লেখার কারণে।

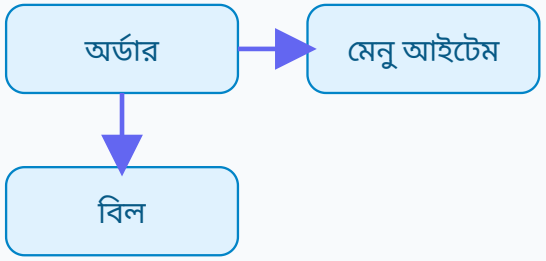
শিক্ষকের নোট: এই স্লাইডে বলা "৯টি" = ১২ − ৩ (IMS + রেস্টুরেন্ট POS + যেটা আপনি টাই-ব্রেকে রাখবেন)। শ্রেণিকক্ষে সংখ্যা বদলাতে পারেন, কারণটা বদলাবেন না।

ধাপ ৪

টাই ভাঙল কীভাবে — স্কুলের ডেটার আকৃতি

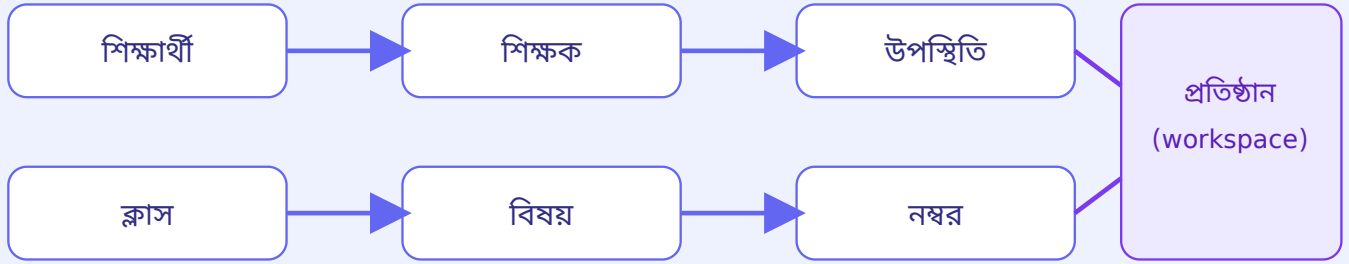
৯ বনাম ৯: রেস্টুরেন্ট আর স্কুলের মধ্যে তফাত কোথায়

রেস্টুরেন্ট POS



এক দিকের স্রোত · ভূমিকা ২টি

IMS — স্কুল



একই শিক্ষার্থী বহু ক্লাসে · বহু শিক্ষক · ভূমিকা ৫টি · সবকিছু প্রতিষ্ঠানভিত্তিক

POS-এ সম্পর্ক একমুখী — অর্ডার → বিল। স্কুলে সম্পর্ক **জালের মতো**: এক শিক্ষার্থী বহু ক্লাসে, এক ক্লাসে বহু শিক্ষক, এক শিক্ষক বহু প্রতিষ্ঠানে। এই জালটাই আপনাকে শেখাবে **foreign key, many-to-many, এবং tenant isolation** — তিনটে সবচেয়ে দামি ধারণা।

শিক্ষকের নোট: ছাত্রদের বলুন — "যে প্রজেক্টে টেবিলের মধ্যে তির ছোড়া যায়, সেটা নাও। যে প্রজেক্টে একটাই টেবিল — সেটা টিউটোরিয়াল, প্রজেক্ট নয়।"

ধাপ ৫

"স্কোপ ফ্রিজ" — এক পাতায় তিনটি বাক্স

আজকের কাজ শেষ হলে এই তিনটি বাক্স ভরে ফেলুন, তারপর বরফে ঢালুন

🧊 MVP — ফ্রিজে

কোর্স শেষে **অবশ্যই** চলবে। কম নয়, বেশি নয়।
IMS-এ: লগইন · প্রতিষ্ঠান · ছাত্র · উপস্থিতি · নম্বর · ড্যাশবোর্ড

🕐 v1.1 — পরে

দরকার, কিন্তু ফ্রিজের বাইরে। তারিখ লিখে রাখুন।
IMS-এ: আমন্ত্রণ · ব্যাকআপ · রিপোর্ট এক্সপোর্ট · নোটিফিকেশন

🚫 কখনো নয়

সচেতনভাবে বাদ। কারণ লিখে রাখুন, নইলে তিন মাস পর ফিরে আসবে।
IMS-এ: SMS গেটওয়ে · AI উপস্থিতি · মোবাইল অ্যাপ

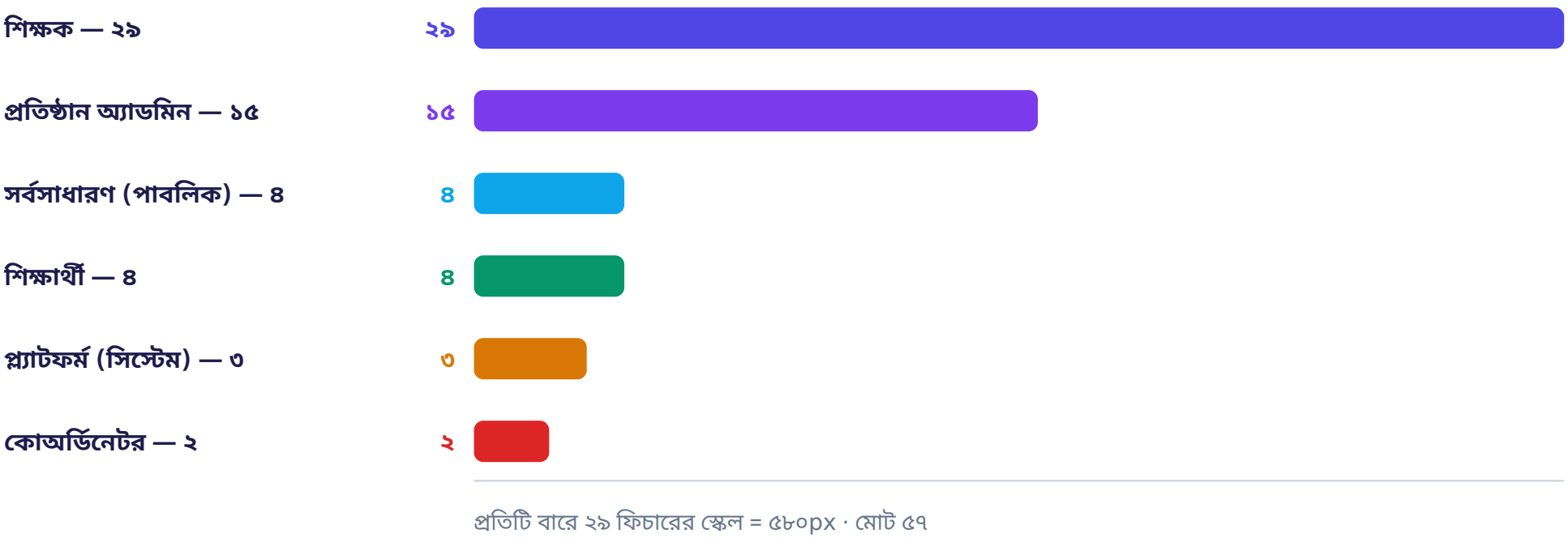
ফ্রিজের নিয়ম একটাই: **নতুন ফিচার যোগ করতে হলে অন্য একটা ফিচার বাদ দিতে হবে।** স্কোপ বাড়ে না — শুধু বদলায়।

শিক্ষকের নোট: "কখনো নয়" বাক্সটা সবচেয়ে জরুরি আর সবচেয়ে বেশি বাদ পড়ে। বোর্ডে লিখে রাখুন — সেমিস্টারের মাঝে কেউ ওই ফিচার চাইলে এই লেখাটাই আপনার উত্তর।

ধাপ ৬

৫৭টি ফিচার, ৬টি ভূমিকা — ফ্রিজ করা তালিকা

প্রজেক্টের আসল ডকুমেন্ট: `docs/catalogue_data.py` → F-01 … F-57



> প্রথমেই ভাববেন না "কী বানাব" — ভাবুন **"কে ব্যবহার করবে, আর সে কী করতে পারবে"**। ভূমিকা আগে, পর্দা পরে।

> **শিক্ষকের নোট:** জিজ্ঞেস করুন — "শিক্ষকের ২৯, শিক্ষার্থীর ৪। এটা কি অন্যায়?" উত্তর: না — শিক্ষক হচ্ছেন সিস্টেমের *উৎপাদক*, শিক্ষার্থী *ভোক্তা*। যে লেখে তার কাজ বেশি, যে পড়ে তার কাজ কম।

ধাপ ৭

একটি ফিচার দেখতে কেমন — F-31, হুবহু প্রজেক্ট থেকে

এই ছয়টি লাইন না লিখলে ফিচারটা "ধারণা", "স্পেসিফিকেশন" নয়

```
F-31  উপস্থিতি নেওয়া
ভূমিকা : TEACHER
রুট     : /w/<id>/attendance/take/
উদ্দেশ্য: এক দিনের জন্য পুরো ক্লাসকে
          একবারে উপস্থিত/অনুপস্থিত/
          দেরি/ছুটি চিহ্নিত করা
ক্রিয়া  : ক্লাস, তারিখ, বিষয় বাছুন
          bulk বোতামে সবাইকে চিহ্নিত করুন
          নোট যোগ করুন → সেভ
ডেটা    : AttendanceRecord unique(
          student, class, subject, date)
          লকড টার্মে তারিখ froze
অবস্থা  : আজ নেওয়া হয়ে গেছে → edit mode
          ভবিষ্যৎ তারিখ বন্ধ
          ছুটির দিনে সতর্কবার্তা
          কোনো ছাত্র নেই
```

ছয়টি লাইন = ছয়টি প্রশ্নের উত্তর

ভূমিকা — কে ঢুকতে পারবে?
রুট — URL কী? (তাই permission যাচাই করা যায়)
উদ্দেশ্য — এক বাক্যে কাজটা
ক্রিয়া — পর্দায় ঠিক কী কী বোতাম
ডেটা — কোন টেবিল, কোন unique নিয়ম
অবস্থা — ভুল হলে কী দেখাবে

"অবস্থা" লাইনটা বাদ দিলেই ৮০% বাগ জন্মায় — খালি টেবিল, ভবিষ্যতের তারিখ, ডবল সাবমিট।

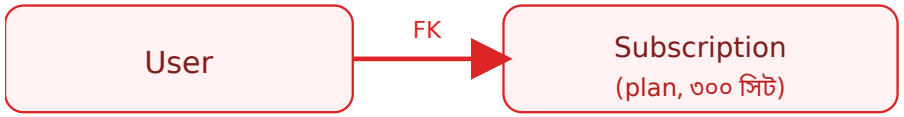
শিক্ষকের নোট: বোর্ডে নিজের প্রজেক্টের একটা ফিচার এই ছয় লাইনে লিখিয়ে নিন। যে ছাত্র "ডেটা" লাইনটা লিখতে পারবে না, তার স্কোপ এখনো ফ্রিজ হয়নি — ক্লাস ১৪-এ তার ডেটাবেস ডিজাইন ভাঙবে।

ধাপ ৮

ফ্রিজ ভাঙার আসল ঘটনা — সাবস্ক্রিপশন ভুল জায়গায় বাঁধা ছিল

কমার্শিয়াল লেয়ার দেরিতে যোগ হয় → টেবিলের সম্পর্ক উল্টে যায়

ভুল (প্রথম লেখা)



শিক্ষক ২ স্কুলে পড়ান → কার প্ল্যান? ব্যক্তির নামে বিল, প্রতিষ্ঠানের নয়।

ঠিক (commit 8a25724)



বিল প্রতিষ্ঠানের নামে · সিট প্রতিষ্ঠানের
শিক্ষক শুধু সদস্য

যা শিখতে হবে: স্কোপ ফ্রিজ শুধু "কী বানাব" ঠিক করে না — **কে কার অধীন** সেটাও ঠিক করে। এই একটি সিদ্ধান্ত ভুল হলে টেবিল, ফর্ম, পারমিশন, রিপোর্ট — সব একসাথে বদলাতে হয়।

শিক্ষকের নোট: প্রশ্ন করুন — "এক শিক্ষক দুই প্রতিষ্ঠানে পড়ান। তার এক প্রতিষ্ঠানের সাবস্ক্রিপশন শেষ। সে কি অন্য প্রতিষ্ঠানে ঢুকতে পারবেন?" সঠিক উত্তর: হ্যাঁ — কারণ বিল ব্যক্তির নয়, প্রতিষ্ঠানের।

ধাপ ৯

একটা ফিচারের দাম কত — বাস্তব হিসাব

IMS-এর ১১টি অ্যাপ থেকে গোনা সংখ্যা

🗄️

৩১

ডেটাবেস টেবিল · ২৩২টি ফিল্ড · ১২টি মাইগ্রেশন

🖼️

৭৫

টেমপ্লেট — অর্থাৎ প্রতি ফিচারে ~১.৭টি পর্দা

🔗

১১

urls.py · এক ফিচার = অন্তত ১টি রুট + ১টি ভিউ

🧪

১০৩

টেস্ট — প্রতি ফিচারে গড়ে ১.৮টি স্বয়ংক্রিয় পরীক্ষা

ফিচারের ধরন	যা লাগে	আনুমানিক সময়
তালিকা পর্দা (list)	মডেল + ভিউ + ১ টেমপ্লেট + ১ টেস্ট	আধা দিন
ফর্ম পর্দা (create/edit)	ফর্ম + ভ্যালিডেশন + ৪টি error অবস্থা	১ দিন
রিপোর্ট / হিসাব	কোয়েরি + অ্যাগ্রিগেট + এক্সপোর্ট	২ দিন
নতুন ডেটা মডেল	মাইগ্রেশন + অ্যাডমিন + পুরনো ডেটার যত্ন	৩ দিন

শিক্ষকের নোট: এই টেবিলটা মুখস্থ করাতে হবে না — ব্যবহার করাতে হবে। ছাত্র "আরেকটা ফিচার যোগ করব" বললে বলুন: "ঠিক আছে, কোন ধরনের? কত দিন? কোনটা বাদ দেবে?"

ধাপ ১০

স্কোপ সত্যিই কতটা শেষ — সংখ্যাটা সৎভাবে বলুন

৫৭টি ফ্রিজ করা ফিচার · প্রমাণ হিসেবে স্ক্রিনশট গোনা হয়েছে

৪৩

স্ক্রিনশট প্রমাণসহ তৈরি

`docs/build_status.py`

১১

কোড আছে, ছবি নেই

প্রতিটির নিজের কারণ লেখা — F-06 · F-32 · F-53 · F-35 · F-15 · F-14 · F-23 · F-10 · F-03 · F-04 · F-07

৩

সত্যিই বাকি

F-11 আমন্ত্রণ · F-12 আমন্ত্রণ গ্রহণ · F-57 ব্যাকআপ কনসোল

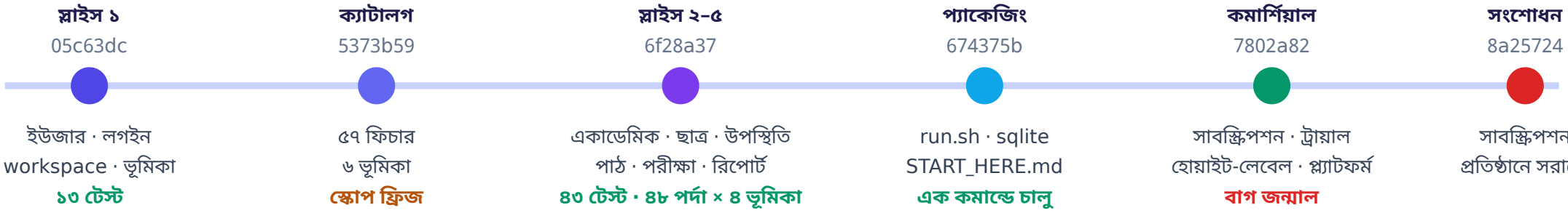
অবস্থা	কী বলবেন ক্লাসে / ডেমোতে	কী করবেন
৪৩ — স্ক্রিনশটসহ	"চালু আছে, স্ক্রিনশট দেখুন"	ডেমোতে দেখান
১১ — ছবি নেই	"কোড আছে ও টেস্টে যাচাই হয়েছে, শুধু পর্দার ছবি নেই"	টেস্টের নাম দেখান
৩ — বাকি	"পরিকল্পিত, হয়নি — কারণ এই"	v1.1 তালিকায় রাখুন

শিক্ষকের নোট: এটাই কোর্সের সবচেয়ে সৎ স্লাইড। "সব হয়ে গেছে" বললে এক প্রশ্নেই ধরা পড়বে। "৪৩টার স্ক্রিনশট আছে, ১১টা টেস্টে যাচাই হয়েছে কিন্তু ছবি নেই, ৩টা সত্যিই বাকি" — এই উত্তর আপনাকে বিশ্বাসযোগ্য করে।

ধাপ ১১

বড় কাজকে স্লাইসে ভাগ করা — আসল কমিটগুলোর ক্রম

প্রতিটি স্লাইস শেষে সিস্টেম চলে, টেস্ট সবুজ



নিয়ম: প্রতিটি স্লাইস এমন একটা সিস্টেম রেখে যায় যা চলে, এবং প্রতিটি স্লাইসের শেষে টেস্ট সবুজ থাকে।
এক সপ্তাহে "সব কিছু" বানাতে গিয়ে কিছুই চলে না — এই ফাঁদেই পড়ে ৮০% প্রজেক্ট।

শিক্ষকের নোট: ভার্সিটি দেখিয়ে বলুন — ক্যাটালগ (স্লাইস ১.৫) স্লাইস ২–৫ এর *আগে* লেখা হয়েছিল। প্ল্যান আগে, কোড পরে। এখানেই "স্কোপ ফ্রিজ" শব্দটার মানে বাস্তবে দেখা যায়।

ধাপ ১২

"আরও একটা ছোট ফিচার" — এই বাক্যের দাম

ফিচার ক্রিপ: দেরিতে যোগ হওয়া ফিচার যখন পুরনো ডিজাইন ভাঙে

🧨 যা হয়

কমার্শিয়াল লেয়ার যোগ হলো **স্লাইস ২–৫ শেষে**। তখন ৩১ টেবিল দাঁড়িয়ে গেছে। সাবস্ক্রিপশন ব্যক্তির নামে বাঁধায় প্রতিষ্ঠানের ধারণাটাই ভুল হয়ে গেল।

ঠিক করতে হয়েছে: ১টি মাইগ্রেশন · বিলিং ভিউ · প্ল্যাটফর্ম কনসোল · ৪টি টেস্ট।

✅ যা করা উচিত ছিল

প্রথম দিনেই একটা লাইনে ঠিক করা: **"টাকার একক = প্রতিষ্ঠান, মানুষ নয়।"** এক মিনিটের সিদ্ধান্ত, তিন দিনের কাজ বাঁচাত।

ফিচার ক্রিপের তিনটি চেহারা

১. **"শুধু একটা বোতাম"** — পুরনো ফর্মে নতুন চেকবক্স → ভ্যালিডেশন ভাঙে।
২. **"শুধু একটা রিপোর্ট"** — নতুন কোয়েরি → নতুন ইনডেক্স → নতুন টেবিল।
৩. **"শুধু একটা ভূমিকা"** — ৫ ভূমিকা থেকে ৬ → **সব** পারমিশন আবার যাচাই।

প্রতিরক্ষা: নতুন ফিচার এলে **প্রথমে লিখুন কোন ফিচার বাদ যাচ্ছে**, তারপর কোড লিখুন।

শিক্ষকের নোট: এখানে বাস্তবের সবচেয়ে অনুভূত অংশটা বলুন — "একটা ফিচার যোগ করার সবচেয়ে বড় খরচ কোড লেখা নয়, সব পুরনো জিনিস আবার যাচাই করা।"

আলোচনা

চারটি প্রশ্ন — জোড়ায় জোড়ায় ৫ মিনিট

উত্তর বোর্ডে লিখুন, আজকের শেষে ফিরে আসব

১. আপনার প্রজেক্টের স্কোর কার্ডে "প্রমাণ দেখানো যাবে?" ঘরে কত দিলেন? ঠিক কী স্ক্রিনশট দিয়ে প্রমাণ করবেন, সেটা কি আজই বলতে পারবেন?

২. আপনার MVP-তে কতটি ফিচার? ১৪ দিনের কাজ ফেলে দিলে কোনটা সবচেয়ে আগে বাদ যাবে?

৩. আপনার সিস্টেমে "টাকার একক" কী — ব্যক্তি, প্রতিষ্ঠান, নাকি অর্ডার? এক লাইনে লিখুন।

৪. যে ফিচারটা আপনি সবচেয়ে ভালোবাসেন — সেটার প্রমাণ কি আজ দেখাতে পারবেন? না পারলে সেটা ভালোবাসা, ফিচার নয়।

শিক্ষকের নোট: ৪ নম্বর উত্তরটা সবচেয়ে যন্ত্রণার আর সবচেয়ে দরকারি। যারা "পারব না" বলবে, তাদের কাছ থেকেই সেরা প্রজেক্টগুলো আসে — কারণ তারা সত্যি বলছে।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
ভালো লাগা দিয়ে প্রজেক্ট বাছা যায় না — ৫ মাপকাঠির স্কোর কার্ড লাগে	১২টি আইডিয়ার টেবিলে দুইটি ৯ পেয়েছিল; টাই ভাঙল ডেটার জাল
স্কোপ ফ্রিজ = তিনটি বাক্স (MVP / পরে / কখনো নয়)	৫৭ ফিচারের তালিকা `catalogue_data.py`-তে ফ্রিজ করা
ফিচার লিখতে হয় ছয় লাইনে — ভূমিকা, রুট, উদ্দেশ্য, ক্রিয়া, ডেটা, অবস্থা	F-31 হুবহু প্রজেক্ট ফাইল থেকে
সৎ সংখ্যা বলাই শক্তি	৪৩ স্ক্রিনশটসহ + ১১ টেস্টে যাচাই + ৩ বাকি = ৫৭
দেরিতে যোগ হওয়া ফিচার পুরনো ডিজাইন ভাঙে	কমার্শিয়াল লেয়ার (7802a82) → সাবস্ক্রিপশন সংশোধন (8a25724)

শিক্ষকের নোট: ছাত্রদের বলুন এই পাঁচ লাইন তাদের প্রজেক্ট ডায়েরির প্রথম পাতা হিসেবে লিখে রাখতে — সেমিস্টার শেষে "কী শিখলাম" প্রশ্নের উত্তর এখানেই তৈরি হয়ে যাবে।

০৩

পরের ক্লাস — "এটা কি সত্যিই সম্ভব?"

ফিজিবিলিটি স্টাডি: প্রযুক্তি · পরিচালনা · অর্থ · সময় — আর ভাঙার বিন্দুর হিসাব

পর্ব ১ · কনসেপ্ট · ক্লাস ৩ / ৩৭

এটা কি সত্যিই সম্ভব?

সিলেবাস: প্র্যাকটিক্যাল ২ · টপিক ২.১ (ফিজিবিলিটি স্টাডি)

চারটি দিক: প্রযুক্তি · পরিচালনা · অর্থ · সময়

৯,৩২২ লাইন Python · ২ দিন · ভাঙার বিন্দু ২ গ্রাহক

শুরু করুন প্রশ্ন দিয়ে

ফিজিবিলিটি মানে "পারব কি না" — এটা ভুল ধারণা

হাত তুলুন: আপনার প্রজেক্ট "সম্ভব" — কীসের ভিত্তিতে বলছেন?

"পারব" একটা অনুভূতি।
ফিজিবিলিটি একটা হিসাব — চারটি আলাদা খাতায়।

চারটির যেকোনো একটিতে লাল বাতি জ্বললে প্রজেক্টটা মরে — বাকি তিনটি সবুজ হলেও। আজ প্রতিটি খাতায় **আসল সংখ্যা** বসাব, নিজের অনুমান নয়।

শিক্ষকের নোট: কেউ "সবই পারব" বললে জিজ্ঞেস করুন — "মাসে সার্ভার খরচ কত পড়বে? কে দেবে?" উত্তর না এলে বলুন: "তাহলে এটা এখনো ফিজিবিলিটি নয়, ইচ্ছা।"

ধাপ ১

চারটি খাতা — প্রতিটির নিজস্ব প্রশ্ন

প্রতিটি খাতায় একটি প্রশ্ন, একটি প্রমাণ, একটি লাল বাতির শর্ত

⚙️ প্রযুক্তি

প্রশ্ন: এই কাজটা করার মতো উপকরণ আছে?

প্রমাণ: **Django 6.1.2 · Python 3.13 · ৬টি প্যাকেজ**

লাল বাতি: যে কাজটা লাইব্রেরি ছাড়া করতে হবে

🔧 পরিচালনা

প্রশ্ন: চালু রাখবে কে, কীভাবে?

প্রমাণ: **deploy_check.sh · ৩টি settings · ২ worker**

লাল বাতি: "শুধু আমার ল্যাপটপে চলে"

💰 অর্থ

প্রশ্ন: খরচ কত, আয় কত, কোথায় ভাঙবে?

প্রমাণ: **৳২,১০০/মাস খরচ · ভাঙার বিন্দু ২ গ্রাহক**

লাল বাতি: খরচের হিসাবই নেই

⏳ সময়

প্রশ্ন: কার কত সময় লাগবে?

প্রমাণ: **২ কর্মদিবস · ২০টি কমিট · ১০৩ টেস্ট**

লাল বাতি: শেষ ২ সপ্তাহে সব পেন্ডিং

আজকের ক্লাসে চারটি খাতার প্রতিটিতে **আসল সংখ্যা** বসাব — প্রজেক্টের ফাইল থেকে তোলা।

শিক্ষকের নোট: বোর্ডে চারটি বাক্স আঁকুন, প্রতি বাক্সে একটি প্রশ্ন লিখুন। ছাত্রদের বলুন নিজের প্রজেক্টের জন্য একই চারটি বাক্স খাতায় আঁকতে — ক্লাস ৪-এর কনসেপ্ট পেপারে এটাই যাবে।

খাতা ১

প্রযুক্তি — আটটি নাম, ছয়টি আসলে ব্যবহৃত

প্রজেক্টের `requirements.txt` হুবহু — তারিখ ও ভার্সনসহ

উপকরণ	ভার্সন	কী কাজে	না থাকলে কী হতো
Python	৩.১৩	ভাষা — সব কোড এর ওপর দাঁড়িয়ে	—
Django	৬.১.২	লগইন, পারমিশন, ORM, ফর্ম, অ্যাডমিন	নিজে লিখতে হতো ~২,০০০ লাইন
psycopg	৩.৩.৬	PostgreSQL-এর সাথে কথা বলা	sqlite-এ ৮০০ সিট আটকাবে না
python-dotenv	১.২.৪	গোপন তথ্য `.env` ফাইলে রাখা	পাসওয়ার্ড কোডে ঢুকে যেত
Pillow	১২.৩.০	লোগো/প্রতিষ্ঠানের ব্র্যান্ডিং ছবি	লোগো আপলোড ভাঙত
openpyxl	৩.১.৫	Excel (.xlsx) এক্সপোর্ট	CSV-তে বাংলা ভাঙত
reportlab	৫.১.০	মার্কশিট PDF	ব্রাউজারের প্রিন্ট ভরসা করতে হতো
gunicorn	শুধু prod	আসল সার্ভারে চালানোর জন্য	dev server দিয়ে prod নয়

নিয়ম: **প্রতিটি প্যাকেজের জন্য এক লাইনে লিখতে হবে "না থাকলে কী হতো"**। যে প্যাকেজের উত্তর দিতে পারেন না — সেটা বাদ দিন।

শিক্ষকের নোট: ভার্সন পিন করা কেন — জিজ্ঞেস করুন। উত্তর: "আজ যা চলছে, ৬ মাস পরে আবার ঠিক তাই চলবে।" পিন না করলে যেকোনো একটা আপডেট সব ভেঙে দেয়।

খাতা ১

চারটি বিকল্প, একটা বাছাই — কেন Django

প্রযুক্তি বাছাই = স্বাদ নয়, প্রয়োজন

বিকল্প	লগইন ও ভূমিকা	ডেটাবেস	ডেটা জটিলতা	ডাকুমেন্টেশন	মোট /৮	সিদ্ধান্ত
Django	বিল্ট-ইন (২)	ORM + মাইগ্রেশন (২)	৩১ টেবিল সহজে (২)	অসাধারণ (২)	**৮**	✓ **বাছা হলো**
Flask	নিজে লিখতে হবে (০)	SQLAlchemy যোগ (১)	সম্ভব, কিন্তু কাঠামো নেই (১)	ভালো (২)	৪	○ বাদ
Node + Express	নিজে (০)	Prisma/Sequelize (১)	সম্ভব (২)	ভালো (২)	৫	○ ভাষা বদলাতে হতো
PHP (Laravel)	আছে (২)	Eloquent (২)	সম্ভব (২)	ভালো (২)	৮	◐ সমান, কিন্তু কোর্স Python
No-code (Airtable ইত্যাদি)	আছে (২)	সীমিত (১)	সীমিত — কোড নেই (০)	সহজ (২)	৫	✕ শেখার উদ্দেশ্যই ভাঙে

শিক্ষকের সিদ্ধান্তের কারণ: কোর্স Python-ভিত্তিক, আর Django-তে **পারমিশন আর ভূমিকা** নিজে বানাতে হয় না — শিখতে হয়।

সতর্কতা: Laravel-ও ৮ পেয়েছে। "সেরা প্রযুক্তি" বলে কিছু নেই — যেটা **শেখানো হচ্ছে** আর যেটা **আপনি জানেন**, সেটাই সেরা।

শিক্ষকের নোট: ছাত্র "রিঅ্যাক্ট + Node" বললে বাধা দেবেন না, কিন্তু প্রশ্ন করুন: "লগইন, ভূমিকা, অডিট লগ — এই তিনটে কে লিখবে? কত দিনে?"

খাতা ১ · প্রমাণ

স্কেলের হিসাব — ৮০০ সিট মানে কত লাইন ডেটা

এখানেই sqlite আর PostgreSQL-এর পার্থক্য তৈরি হয়

👥

৩০০ শিক্ষার্থী

শিক্ষকদের আটেনড্যান্স দৈনিক একবার

→ বছরে ৬০,০০০ সারি

📚

দিনে ৬ পিরিয়ড

প্রতি পিরিয়ডে আলাদা হাজিরা

→ বছরে ৩,৬০,০০০ সারি

🏫

২০ প্রতিষ্ঠান

একই সার্ভারে ২০টি স্কুল

→ বছরে ৭২ লাখ সারি

প্রশ্ন	sqlite	PostgreSQL
একসাথে কয়জন লিখতে পারে?	প্রায় একজন (লেখার সময় লক)	অনেকজন, সারি-পর্যায়ে লক
৭২ লাখ সারিতে রিপোর্ট	ধীরে, ইনডেক্স সীমিত	ইনডেক্স + partial index + EXPLAIN
নিরাপত্তা / ব্যবহারকারী আলাদা করা	ফাইল অনুমতি	রোল, গ্রান্ট, স্কিমা
ব্যাকআপ	ফাইল কপি	pg_dump, PITR

প্রজেক্টে তাই **তিনটি settings**: `dev` (sqlite, দ্রুত শুরু) · `sqlite` (এক-ক্লিকে ডেমো) · `prod` (PostgreSQL + gunicorn)।

শিক্ষকের নোট: সংখ্যাগুলো বোর্ডে ধীরে ধীরে গুণ করুন — ৩০০ × ২০০ = ৬০,০০০। ছাত্ররা তখনই বুঝবে "আমার ছোট ডেটা" বলে কিছু নেই; সেমিস্টার শেষে ডেটা বড় হয়।

খাতা ২

পরিচালনা — "আমার ল্যাপটপে চলে" যথেষ্ট নয়

প্রমাণ: `docs/deploy_check.sh` — প্রোডাকশনের মহড়া

```
$ bash docs/deploy_check.sh
-> collectstatic
-> gunicorn on :8010 (DEBUG=False)
   --workers 2
-> checks
  login page            200
  static css            200
  unknown page (404)    404
  workspace page anon   302
-> security headers
  X-Content-Type-Options: nosniff
  X-Frame-Options: DENY
  Referrer-Policy: same-origin
-> done (server stopped)
```

চারটি সংখ্যা যাচাই হলো

200 — পর্দা খুলছে

200 — CSS লোড হচ্ছে (না হলে সাদা পাতা)

404 — ভুল ঠিকানায় ভদ্র উত্তর

302 — অচেনা লোক প্রতিষ্ঠানের ভেতরে ঢুকতে পারছে না

প্রোডাকশনে `DEBUG=True` থাকলে ভুল হলেই সোর্স কোড, পাসওয়ার্ড, টেবিলের নাম — সব browser-এ দেখা যায়।

শিক্ষকের নোট: ৩০২ কেন ৪০৪ নয় — এটাই আজকের সবচেয়ে দামি প্রশ্ন। উত্তর: "ঢুকতে দিই না" আর "কিছু নেই" আলাদা কথা। ৩০২ মানে "আগে লগইন করুন"।

খাতা ৩

অর্থ — মাসিক খরচের সৎ হিসাব

বাংলাদেশের বাজারদরে, ২০২৬ · সংখ্যাগুলো অনুমান, পদ্ধতিটা আসল

খরচের খাত	মাসিক	ব্যাখ্যা
VPS (২ GB RAM, ২ vCPU)	৳১,৮০০	৳২২,০০০/বছর — Django + PostgreSQL একসাথে
ডোমেইন + SSL	৳১০০	SSL বিনামূল্যে (Let's Encrypt), ডোমেইন ৳১,২০০/বছর
ব্যাকআপ স্টোরেজ	৳২০০	প্রতিদিনের dump, ৩০ দিন রাখা
ইমেইল (SMTP)	৳০	ফ্রি টিয়ার দিয়ে শুরু
মোট নির্দিষ্ট খরচ	**৳২,১০০**	গ্রাহক শূন্য হলেও কাটবে
পেমেন্ট গেটওয়ে ফি	২.৫%	প্রতি লেনদেনে — নির্দিষ্ট খরচ নয়
সাপোর্ট (প্রতি গ্রাহক)	৳২৫০	মাসে ২–৩টি ফোন/মেসেজের সমতুল্য

যে খরচটা সবাই ভুলে যায়: **সাপোর্ট**। সফটওয়্যার বিক্রি একবার, ফোন আসে প্রতি সপ্তাহে। এই লাইনটা না লিখলে হিসাব সবসময় মিথ্যা দেখায়।

শিক্ষকের নোট: নিজের শহরের আসল দাম জিজ্ঞেস করুন — কেউ জানলে বোর্ডে লিখুন। সংখ্যা বদলালে সমস্যা নেই, পদ্ধতি বদলালে সমস্যা।

খাতা ৩ · হিসাব

ভাঙার বিন্দু — কতজন গ্রাহকে খরচ উঠে যায়

প্রতি গ্রাহক থেকে হাতে কত থাকে, তারপর ভাগ

Starter — ৳১,৫০০/মাস

আয় ১,৫০০ | −৩৭ গেটওয়ে ২.৫% | −২৫০ সাপোর্ট | **হাতে থাকে ১,২১৩** → ২ জনে খরচ উঠে যায়

Growth — ৳৩,৫০০/মাস

আয় ৩,৫০০ | −৮৮ | −৪০০ সাপোর্ট | **হাতে থাকে ৩,০১২** → ১ জনেই খরচ উঠে যায়

নির্দিষ্ট খরচ ৳২,১০০/মাস ÷ প্রতি গ্রাহকের নিট আয় = ভাঙার বিন্দু

Starter: ২,১০০ ÷ ১,২১৩ = ১.৭ → ২ জন · Growth: ২,১০০ ÷ ৩,০১২ = ০.৭ → ১ জন

> এই একটি হিসাবই বলে দেয়: প্রজেক্টটা **টিকতে ২ জন গ্রাহক লাগবে**। "লাখ লাখ ব্যবহারকারী" নয় — মাত্র দুইজন।

শিক্ষকের নোট: ছাত্রদের নিজের প্রজেক্টে এই ভাগটা করাতে হবে। যাদের উত্তর "কোনো খরচ নেই" আসবে, তাদের বলুন — ডোমেইন, হোস্টিং, ব্যাকআপ, নিজের সময়: অন্তত একটি আছে।

খাতা ৩ · প্রমাণ

দামগুলো কোথা থেকে — তিনটি প্ল্যান, আসল কোড থেকে

`apps/billing/management/commands/seed_plans.py`

প্ল্যান	দাম	শিক্ষার্থী সীমা	স্টাফ সীমা	কার জন্য
Starter	৳১,৫০০/মাস	৩০০	২৫	এক ক্যাম্পাস, ছোট স্কুল
Growth	৳৩,৫০০/মাস	৮০০	সীমা নেই	বড় স্কুল, অনেক শিক্ষক
Campus	৳৬,৫০০/মাস	সীমা নেই	সীমা নেই	মাল্টি-ব্রাঞ্চ + নিজের ডোমেইন

সীমাটা আসে কোথা থেকে?

সীমা কোডে `max_students` ফিল্ডে থাকে — `null` মানে সীমা নেই। তাই নতুন প্ল্যান যোগ করতে **কোড বদলাতে হয় না**, শুধু ডেটা।

কেন ৩০০, ৮০০, ∞?

প্রতিটি সীমা একটা **সিদ্ধান্তের বেড়া**: ৩০০-এর নিচে একজন অ্যাডমিন হাতে সামলাতে পারে; ৮০০ ছাড়ালে আলাদা কোঅর্ডিনেটর দরকার হয়।

ভুলটা কী ছিল

সাবস্ক্রিপশন **ব্যক্তির** নামে বাঁধা ছিল। প্রতিষ্ঠানের সিট কে ব্যবহার করছে — সেই প্রশ্নের উত্তরই ছিল না।

শিক্ষকের নোট: প্রশ্ন করুন — "৩০০ সিটের স্কুল ৩০১-তম ছাত্র ভর্তি করলে কী হবে?" সঠিক উত্তর: সিস্টেম আটকাবে, আর সেই আটকানোটা কোম্পানির আয় — এটা নিষ্ঠুর শোনালেও এটাই ব্যবসার নিয়ম, তাই সীমাটা কোডে নয়, প্ল্যানে থাকে।

খাতা ৪ · প্রমাণ

সময় — আসল টাইমলাইন, অনুমান নয়

২০টি কমিট, ২ কর্মদিবস · `git log` থেকে

দিন ১ — ২৯ অক্টোবর, সকাল

০৬:২১ স্লাইস ১ · ০৬:২৪ CSRF ফিক্স · ০৬:২৮ স্ক্রিনশট
০৬:৩৬ ৫৭ ফিচারের ক্যাটালগ · ০৬:৫২ স্লাইস ২–৫
০৭:০৮ প্যাকেজিং · ০৮:১০ UI সংশোধন

৯ কমিট

৬ দিন

দিন ২ — ৪ নভেম্বর, দুপুর

১৩:২৩ প্রোডাকশন হার্ডেনিং · ১৩:৩৯ কমার্শিয়াল লেয়ার
১৪:০৪ বিলিং বাগ ফিক্স · ১৫:০২ workspace ফিক্স
১৫:৪০–১৭:১২ রিনেম ও ডকুমেন্টেশন

১১ কমিট

৯,৩২২
লাইন Python
(মাইগ্রেশন বাদ দিয়ে)

৩,২৬৫
লাইন HTML (৭৫ টেমপ্লেট)
স্ক্রিনশট প্রমাণসহ

×৪ – ×৬
ছাত্রের জন্য গুণক
৪–৬ সপ্তাহের সমান কাজ

শিক্ষকের নোট: এই "×৪–×৬" গুণকটাই সবচেয়ে দরকারি কথা। অভিজ্ঞ ডেভেলপারের ২ দিন = ছাত্রের ৮–১২ দিন, আর তার সাথে পড়া, পরীক্ষা, অন্য বিষয়। তাই ১২ সপ্তাহকে ফাঁকা ধরে নয়, ৬ সপ্তাহ ধরে প্ল্যান করতে হয়।

খাতা ৪ · পদ্ধতি

সময়ের হিসাব করার তিনটি ধাপ

যে হিসাব "আশা" দিয়ে হয়, সেটা হিসাব নয়

#	ধাপ	কীভাবে	IMS-এ প্রয়োগ
১	**ভাগ করো**	প্রতিটি ফিচারকে ক্ষুদ্রতম "চলমান" টুকরোয় ভাঙো	৫৭ ফিচার → ৫টি স্লাইস, প্রতিটি নিজে নিজে চলে
২	**গুণ করো**	সবচেয়ে ধীর দিনের গতিতে হিসাব করো, সেরা দিনে নয়	স্লাইস ২–৫: ৪ ঘণ্টার কাজ, ধরা হয়েছিল ১ দিন
৩	**বাদ দাও ৪০%**	যা বাদ দিলেও সিস্টেম চলে — সেটা MVP নয়	আমন্ত্রণ, ব্যাকআপ, PDF এক্সপোর্ট → v1.1

ভুল হিসাব: "৫৭ ফিচার ÷ ১২ সপ্তাহ = সপ্তাহে ৫টি" — কাগজে সুন্দর, বাস্তবে অসম্ভব, কারণ ফিচারগুলো সমান নয়।

ঠিক হিসাব: স্লাইসে ভাগ করো, প্রতিটির জন্য **প্রমাণ** ঠিক করো (টেস্ট সবুজ + স্ক্রিনশট), তারপর যোগ করো।

শিক্ষকের নোট: "৪০% বাদ দাও" নিয়মটা ছাত্ররা প্রথমে মানতে চায় না। বলুন — "যে ফিচারটা বাদ দিলে তুমি কষ্ট পাচ্ছ, সেটাই সবচেয়ে ভালো প্রার্থী। কারণ তুমি এটা ভালোবাসছ, ব্যবহারকারী চায়নি।"

সিদ্ধান্ত

তিনটি বাতি — কখন এগোবেন, কখন থামবেন

ফিজিবিলিটির ফলাফল তিন রঙের বেশি হয় না

🟢

এগিয়ে যান

চার খাতার চারটিতেই সংখ্যা আছে, প্রমাণ আছে, আর MVP এক সপ্তাহে দাঁড়ায়।

IMS: খাতা ৪টিই সবুজ

🟡

ছোট করে এগোন

একটি খাতায় সংখ্যা নেই বা সন্দেহ আছে — স্কোপ কাটুন, তারপর এগোন।

উদাহরণ: পেমেন্ট গেটওয়ে বাদ, ম্যানুয়াল বিলিং

🔴

থামুন

দুই বা তার বেশি খাতায় সংখ্যা নেই — অর্থাৎ এখনো প্রজেক্টটা সংজ্ঞায়িতই হয়নি।

চিহ্ন: "ডেটা পরে ভাবব"

সবচেয়ে বিপজ্জনক অবস্থা **হলুদ** — কারণ তখন সবাই এগোতে চায়, আর কেউ স্কোপ কাটতে চায় না।

শিক্ষকের নোট: হলুদ অবস্থায় স্কোপ কাটার সিদ্ধান্তটা একা নেবেন না — ছাত্রকে নিজে বলতে দিন "কোনটা বাদ দেব"। নিজের মুখে বলা কাটছাঁট সে আর ফিরিয়ে আনবে না।

আলোচনা

চারটি প্রশ্ন — নিজের প্রজেক্টের খাতা মিলিয়ে

প্রতিটির উত্তর একটি সংখ্যা দিয়ে শেষ করতে হবে

১. আপনার প্রজেক্টে বছরে কত সারি ডেটা জমবে? সংখ্যাটা লিখুন — sqlite কি পারবে?

২. মাসিক নির্দিষ্ট খরচ কত? প্রতি গ্রাহকের নিট আয় কত? ভাঙার বিন্দু কতজন?

৩. আপনার সিস্টেম "অন্য কারও সার্ভারে" চালু করতে কী কী লাগবে — তিনটি জিনিস বলুন।

৪. কোন খাতায় আপনার হাতের কাছে *কোনো সংখ্যাই* নেই? ওই খাতাটাই আপনার প্রজেক্টের আসল ঝুঁকি।

শিক্ষকের নোট: ৪ নম্বরের উত্তর দিয়ে ক্লাস ৪ শুরু করবেন — ওখান থেকেই ঝুঁকির তালিকা বের হয়, আর ঝুঁকির তালিকা থেকেই কনসেপ্ট পেপারের ৬টি অংশ।

সারসংক্ষেপ

আজকের পাঁচটি শিক্ষা

প্রতিটি দাবির পাশে তার প্রমাণ

শিক্ষা	প্রমাণ
ফিজিবিলিটি = চারটি খাতার হিসাব, অনুভূতি নয়	প্রযুক্তি · পরিচালনা · অর্থ · সময় — প্রতিটির একটি সংখ্যা
প্রযুক্তি বাছাই প্রয়োজন দিয়ে হয়, স্বাদ দিয়ে নয়	Django ৮/৮ · Flask ৪/৮ · Laravel ৮/৮ কিন্তু কোর্স Python
ছোট ডেটাও এক বছরে বড় হয়	৩০০ শিক্ষার্থী × ৬ পিরিয়ড × ২০০ দিন = ৩,৬০,০০০ সারি
ভাঙার বিন্দু মাত্র দুইজন গ্রাহক	৳২,১০০ নির্দিষ্ট খরচ ÷ ৳১,২১৩ নিট = ১.৭ → ২ জন
অভিজ্ঞ ডেভেলপারের সময় ×৪–×৬	২০ কমিট · ২ কর্মদিবস · ৯,৩২২ লাইন Python

শিক্ষকের নোট: পাঁচটি লাইনেই একটি সংখ্যা আছে — এটাই এই ক্লাসের শর্ত। ছাত্ররা নিজের প্রজেক্টের চারটি সংখ্যা (ডেটা সারি · মাসিক খরচ · ভাঙার বিন্দু · ব্যবধানের গুণক) খাতায় লিখে রাখবে; ক্লাস ৪-এ সেগুলোই কনসেপ্ট পেপারে যাবে।

০৪

পরের ক্লাস — "কী কী ভুল হতে পারে?"

ঝুঁকির তালিকা, আগাম সংকেত, আর কনসেপ্ট পেপারের ছয়টি অংশ

পর্ব ১ · কনসেপ্ট · ক্লাস ৪ / ৩৭

কী কী ভুল হতে পারে?

সিলেবাস: প্র্যাকটিক্যাল ২ · টপিক ২.২–২.৩

ঝুঁকি রেজিস্টার · আগাম সংকেত · কনসেপ্ট পেপারের ছয় অংশ

৬টি ঝুঁকি — প্রতিটি সত্যিই ঘটেছিল

ধাপ ১

ছয়টি ঝুঁকি — প্রতিটির আগাম সংকেত সহ

ঝুঁকি লিখতে হয় তিন কলামে: কী ভুল হতে পারে · আগে থেকে কী দেখে বুঝবেন · কী করবেন

#	ঝুঁকি	আগাম সংকেত	প্রতিকার (প্রজেক্টে যা করা হয়েছে)
১	**মূল ধারণা ভুল বোঝা** কে কার অধীন	"একই প্রশ্নটা দুইবার আসছে"	এক লাইনে লিখে রাখা: **টাকার একক = প্রতিষ্ঠান** (`8a25724`)
২	**পরিবেশ আলাদা** ছাত্রের মেশিনে চলে না	"আমার ল্যাপটপে চলে না"	`run.sh` পাইথন ভার্সন চেক (৩.১২+) + `sqlite` settings (`2cc4e2d`)
৩	**দেরিতে নামকরণ** পরে নাম বদলানো	কোড লেখার *পরে* নাম নিয়ে আলোচনা	নাম `settings`-এ সরানো: `PRODUCT_NAME` (`bfd058e`)
৪	**ডেমো পরিবেশ** লগইন হয়ে আবার লগআউট	প্রিভিউ উইন্ডোতে কুকি থাকে না	কুকি-গার্ড ব্যানার + ভদ্র CSRF পাতা (`e1ead8a`)
৫	**স্কোপ ফুলে যাওয়া**	প্রতিদিন একটা করে নতুন ফিচার	তিন-বাক্স ফ্রিজ + সৎ হিসাব ৪৩/৫৭
৬	**প্রমাণ না রাখা**	"স্ক্রিনশট পরে তুলব"	পরিকল্পনা আর প্রমাণ **আলাদা ফাইল**: `catalogue_data.py` বনাম `build_status.py`

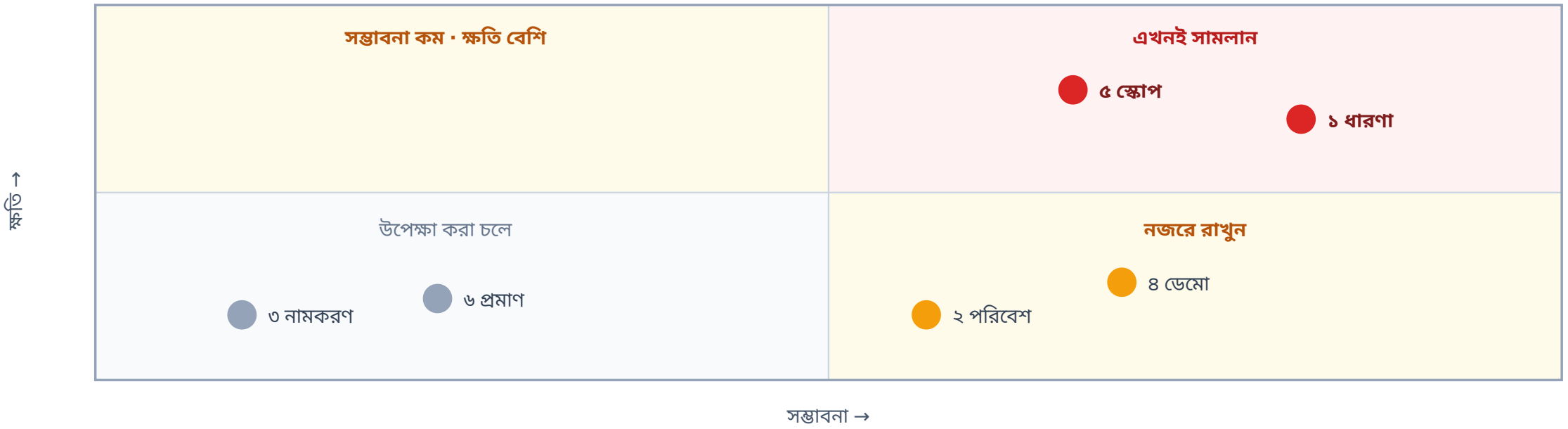
আগাম সংকেত কলামটাই আসল — ঝুঁকি এলে প্রতিকার কাজে লাগে, কিন্তু **সংকেত দেখলে** ঝুঁকিটাই এড়ানো যায়।

শিক্ষকের নোট: এই ছয়টি বানানো ঝুঁকি নয় — প্রতিটির পাশে কমিট নম্বর দেওয়া আছে, অর্থাৎ প্রতিটি সত্যিই ঘটেছিল। ছাত্রদের বলুন নিজের প্রজেক্টে ঠিক এই ছয়টি দিয়ে শুরু করতে; তারপর নিজের মতো দুটি যোগ করতে।

ধাপ ২

কোন ঝুঁকিতে আগে হাত দেবেন — ম্যাপ

সম্ভাবনা × ক্ষতি — উপরে-ডানে থাকলে আগে সামলাতে হবে



সংখ্যাগুলো আগের স্লাইডের ঝুঁকির ক্রম — ১ = ধারণা ভুল বোঝা, ৬ = প্রমাণ না রাখা

১ ও ৫ সবচেয়ে বিপজ্জনক — কারণ এরা ক্ষতি করে **সবচেয়ে দেরিতে** ধরা পড়ে: ধারণা ভুল হলে ৩১ টেবিল ভাঙে, স্কোপ ফুললে শেষ সপ্তাহে।

শিক্ষকের নোট: ছাত্রদের নিজের দুটি ঝুঁকি এই ম্যাপে বসাতে বলুন। সবাই যেটা "উপেক্ষা করা চলে" ঘরে ফেলে, সেটাই সাধারণত পরীক্ষার আগের রাতে ফেটে পড়ে — জিজ্ঞেস করুন কেন ওটা ওখানে ফেলেছে।

ধাপ ৩ · প্রমাণ

যে ঝুঁকিগুলো সত্যিই ঘটেছিল

চারটি ঘটনা, চারটি কমিট — আর প্রতিটির দাম

🍪 ডেমোতে কুকি থাকে না

প্রিভিউ উইন্ডো কুকি রাখতে পারে না → লগইন করে ঢুকলেই আবার লগইন পাতা।

দাম: ১টি আলাদা কমিট (`e1ead8a`) + ভদ্র এরর পাতা

🧾 বিল ভুল জায়গায়

সাবস্ক্রিপশন ব্যক্তির নামে — প্রতিষ্ঠানের সিট কে ব্যবহার করছে, তার উত্তরই নেই।

দাম: ১ মাইগ্রেশন + বিলিং ভিউ + ৪ টেস্ট (`8a25724`)

🏷️ দেরিতে নাম বদল

TMS → IMS। কোডে নাম ছড়িয়ে ছিল; বদলাতে ৩টি কমিট লাগল, তারপর আবার স্ক্রিনশট।

দাম: ৩ কমিট শুধু নামে (`bfd058e` , `7c27e1b` , `86660f3`)

🐍 ছাত্রের পাইথন পুরনো

"চলে না" — কারণ ৩.১০, প্রজেক্ট চায় ৩.১২+। ছয়টি ভিন্ন এরর, একটাই মূল কারণ।

দাম: `run.sh` -এ ভার্সন চেক + স্পষ্ট বার্তা (`2cc4e2d`)

চারটির কোনোটাই "কোডিং ভুল" নয়। চারটিই **সিদ্ধান্ত বা পরিবেশের** ঝুঁকি — তাই এদের ঠিক করতে কোড নয়, **সিদ্ধান্ত** ঠিক করতে হয়েছে।

শিক্ষকের নোট: এখানেই বলুন — "বাগ মেটানো মানে কোড লেখা নয়, আগের সিদ্ধান্ত ভাঙা।" চারটি উদাহরণেই দেখুন: এক লাইন বদলাতে গিয়ে পাঁচ জায়গা বদলাতে হয়েছে।

ধাপ ৪

এখন সব এক পাতায় — কনসেপ্ট পেপারের ছয়টি অংশ

আগের তিন ক্লাসের সব কাজ এই ছয়টি ঘরে ঢোকে

১. সমস্যা ও প্রেক্ষাপট

খাতা কেন যথেষ্ট নয় — তিনটি ক্ষতি

এক লাইনে সমস্যা-বিবৃতি

ক্লাস ১ · স্লাইড ৩

২. ব্যবহারকারী ও ভূমিকা

৫ ভূমিকা · কে কী দেখতে পায় না

ব্যবহারকারী ≠ গ্রাহক

ক্লাস ১ · স্লাইড ৬–৮

৩. স্কোপ

MVP / পরে / কখনো নয়

৫৭ ফিচার, ছয়-লাইনের স্পেক

ক্লাস ২ · স্লাইড ২১–২৩

৪. ফিজিবিলিটি

চারটি খাতা, চারটি সংখ্যা

ভাঙার বিন্দু · সময়ের গুণক

ক্লাস ৩

৫. ঝুঁকি

৬টি ঝুঁকি + আগাম সংকেত

ম্যাপে কোথায় দাঁড়িয়ে

ক্লাস ৪ · এই স্লাইডের আগের তিনটি

৬. ডেলিভারি পরিকল্পনা

৫টি স্লাইস · প্রতিটির প্রমাণ

কোন সপ্তাহে কী সবুজ হবে

ক্লাস ২ · স্লাইড ২৭

নিয়ম: **ছয়টি অংশ, সর্বোচ্চ দুই পাতা।** যেটা দুই পাতায় ধরছে না — সেটা এখনো ভাবা হয়নি, শুধু লেখা হয়েছে।

শিক্ষকের নোট: এই ছয়টি ঘর বোর্ডে আঁকুন, প্রতিটির নিচে ক্লাস নম্বর লিখুন। ছাত্ররা বুঝবে — পেপারটা নতুন কিছু নয়, গত তিন ক্লাসের কাজের সাজানো রূপ।

অংশ ১–২

সমস্যা আর মানুষ — দুই পাতা, কোনও ভূমিকা নয়

লেখার নিয়ম: প্রতিটি বাক্যে একটা সংখ্যা বা একটা নাম থাকতে হবে

১. সমস্যা ও প্রেক্ষাপট

একটি মাধ্যমিক স্কুলে **৩০০ শিক্ষার্থী**, **২৫ শিক্ষক**। উপস্থিতি ও নম্বর খাতায় লেখা হয়। ফলে — মাসিক রিপোর্টে **কয়েক ঘণ্টা** লাগে, এক বছর পেছনের তথ্য **খোঁজা যায় না**, আর কে নম্বর বদলাল তার **কোনো রেকর্ড থাকে না**।

এক লাইনে: তথ্য একবার লেখা হবে, অনেকবার কাজে লাগবে — আর প্রতিটি বদলের দায় থাকবে।

ভুল লেখা: "এটি একটি আধুনিক, ব্যবহারবান্ধব, স্কেলেবল সিস্টেম।" — এতে কোনও সংখ্যা নেই, কোনও নাম নেই, তাই কিছুই বলা হয়নি।

২. ব্যবহারকারী ও ভূমিকা

প্রশাসক	সব প্রতিষ্ঠান, প্ল্যান	ছাত্রের নম্বর দেখে না
অ্যাডমিন	নিজের প্রতিষ্ঠান	অন্য প্রতিষ্ঠান দেখে না
কোঅর্ডিনেটর	নিজের বিভাগ	বিল দেখে না
শিক্ষক	নিজের ক্লাস	অন্য ক্লাসের নম্বর না
শিক্ষার্থী	শুধু নিজের	অন্য কারও কিছু না

গ্রাহক ≠ ব্যবহারকারী: টাকা দেয় প্রতিষ্ঠান, ব্যবহার করে ২৫ জন শিক্ষক।

শিক্ষকের নোট: "ভুল লেখা" বাক্সটা মুখস্থ করান — "আধুনিক, ব্যবহারবান্ধব, স্কেলেবল" এই তিন শব্দ পেলেই লাল পতাকা। কোনও শিক্ষক এভাবে নিজের প্রজেক্টের পরিচয় লিখলে সেটা পেপারের প্রথম বাক্য হতে পারে না।

অংশ ৩–৪

স্কোপ আর সংখ্যা — দুই পাতা

স্কোপে তালিকা, ফিজিবিলিটিতে সংখ্যা — দুইটাই লাগবে

৩. স্কোপ

MVP (৫টি স্লাইস): লগইন ও ভূমিকা · প্রতিষ্ঠান · শিক্ষার্থী ও ভর্তি · উপস্থিতি · নম্বর ও রিপোর্ট

v1.1: আমন্ত্রণ · ব্যাকআপ · পূর্ণ এক্সপোর্ট · নোটিফিকেশন

কখনো নয়: SMS গেটওয়ে · মোবাইল অ্যাপ · AI উপস্থিতি

মোট: ৫৭ ফিচার · ৬ ভূমিকা · ৩১ টেবিল — প্রতিটির ID (F-01 … F-57)

৪. ফিজিবিলিটি — চারটি সংখ্যা

প্রযুক্তি	Django 6.1.2 / Python 3.13
ডেটা	৩০০ × ৬ × ২০০ = ৩,৬০,০০০ সারি/বছর
খরচ	৳২,১০০/মাস → ভাঙার বিন্দু ২ গ্রাহক
সময়	৫ স্লাইস · অভিজ্ঞ ২ দিন · ছাত্র ×৪–×৬

সিদ্ধান্ত: 🟢 এগিয়ে যাওয়া — চার খাতাতেই সংখ্যা আছে, প্রতিটির প্রমাণ আছে।

স্কোপে **সংখ্যা** থাকলে সেটা মাপা যায়; "বড় সিস্টেম" লেখা থাকলে কিছুই মাপা যায় না।

শিক্ষকের নোট: ফিজিবিলিটির ঘরে "সিদ্ধান্ত" লাইনটা বাদ দেবেন না — সংখ্যা থাকলেও সিদ্ধান্ত না লিখলে পেপারটা পড়ে কেউ জানবে না আপনি এগোচ্ছেন কি থামছেন।

অংশ ৫–৬

ঝুঁকি আর পরিকল্পনা — শেষ দুই পাতা

ঝুঁকিতে সংকেত, পরিকল্পনায় তারিখ ও প্রমাণ

৫. ঝুঁকি (৬টি)

১ ধারণা ভুল	"একই প্রশ্ন দুইবার"
২ পরিবেশ	"আমার মেশিনে চলে না"
৩ দেরিতে নাম	কোডের পরে নাম-আলোচনা
৪ ডেমো পরিবেশ	লগইন হয়ে আবার লগআউট
৫ স্কোপ ফোলা	প্রতিদিন নতুন ফিচার
৬ প্রমাণ নেই	"পরে স্ক্রিনশট তুলব"

৬. ডেলিভারি পরিকল্পনা

সপ্তাহ	কী হবে	প্রমাণ
১–২	লগইন · প্রতিষ্ঠান · ভূমিকা	১৩ টেস্ট
৩	৫৭ ফিচারের তালিকা	ক্যাটালগ ফাইল
৪–৭	ছাত্র · উপস্থিতি · নম্বর · রিপোর্ট	৪৩ টেস্ট · ৪৮ পর্দা × ৪ ভূমিকা
৮	প্যাকেজিং · এক কমান্ডে চালু	`run.sh`
৯–১০	প্রোডাকশন মহড়া	`deploy_check.sh`
১১–১২	ডকুমেন্ট · ডেমো	ম্যানুয়াল · স্ক্রিনশট

পরিকল্পনার প্রতিটি সারিতে **"প্রমাণ"** কলাম থাকতেই হবে। যে সারিতে প্রমাণ লেখা যায় না, সেটা কাজ নয় — ইচ্ছা।

শিক্ষকের নোট: এই টেবিলটাই ছাত্রদের নিজের প্রজেক্টে পূরণ করতে দেবেন। সবচেয়ে কঠিন প্রশ্ন: "৮ম সপ্তাহে তোমার হাতে কী প্রমাণ থাকবে?" উত্তর না এলে বাকি পরিকল্পনাও বদলাতে হবে।

ফলাফল

কনসেপ্ট পেপার — এক পাতায় পুরো প্রজেক্ট

এটাই সেই কাগজ যা নিয়ে কেউ প্রশ্ন করলে ৩০ সেকেন্ডে উত্তর দেওয়া যায়

IMS — Institute Management System

কনসেপ্ট পেপার v1.0 · ১ পাতা · ২০২৬

সমস্যা
খাতায় উপস্থিতি ও নম্বর → রিপোর্টে ঘণ্টা, পেছনের তথ্য নেই, দায় নেই।

ব্যবহারকারী
৫ ভূমিকা: প্রশাসক · অ্যাডমিন · কোঅর্ডিনেটর · শিক্ষক · শিক্ষার্থী। গ্রাহক = প্রতিষ্ঠান।

স্কোপ
৫৭ ফিচার · MVP ৫ স্লাইস · v1.1 তে ৪টি · কখনো নয়: SMS, অ্যাপ, AI।

ফিজিবিলিটি
Django 6.1.2 · ৩.৬ লাখ সারি/বছর · ৳২,১০০/মাস · ভাঙার বিন্দু ২ গ্রাহক।

ঝুঁকি
৬টি — ধারণা ভুল, পরিবেশ, দেরিতে নাম, ডেমো কুকি, স্কোপ, প্রমাণ।

পরিকল্পনা
১২ সপ্তাহ · ৫ স্লাইস · প্রতিটির শেষে টেস্ট সবুজ + স্ক্রিনশট।

আজকের অবস্থা: ৫৪/৫৭ ফিচার তৈরি (৪৩টি স্ক্রিনশটসহ) · ১০৩ টেস্ট · ১৯ কমিট · ৩১ টেবিল · ৭৫ টেমপ্লেট

পরীক্ষা: এই পাতাটা **একজন অপরিচিত মানুষকে** দিন — সে কি ৩০ সেকেন্ডে বলতে পারবে কী বানাচ্ছেন, কার জন্য, কত সময়ে?

শিক্ষকের নোট: ক্লাসে সত্যিই এই পরীক্ষাটা করুন — এক ছাত্রের পেপার আরেক ছাত্রকে পড়তে দিন, ৩০ সেকেন্ড গুনুন। পারলে পেপার ভালো, না পারলে পেপার সুন্দর।

ক্লাসের কাজ

নিজের পেপার লেখা — পাঁচটি ধাপ, ২৫ মিনিট

আজ শেষ না হলেও চলবে; কিন্তু প্রতিটি ঘরে অন্তত একটা সংখ্যা থাকতে হবে

১. ছয়টি ঘর আঁকুন

কাগজে ছয়টি বাক্স — আগে ভরবেন না, শুধু আঁকুন। খালি ঘর দেখলে মাথা কাজ করে।

২. সমস্যা এক লাইনে

কে, কতজন, কী হারাচ্ছে — তিনটে তথ্য একটি বাক্যে।

৩. ভূমিকা তালিকা

প্রতিটি ভূমিকার পাশে লিখুন **সে কী দেখতে পাবে না** — এটাই পারমিশনের নকশা।

৪. স্কোপ তিন বাক্স

MVP-তে সর্বোচ্চ ৫টি, "পরে"-তে ৩টি, "কখনো নয়"-তে ২টি।

৫. ঝুঁকি ৬ + সংকেত

আগের স্লাইডের ছয়টি থেকে শুরু করুন, তারপর নিজের দুটি যোগ করুন।

আলোচনার প্রশ্ন (হাতে তুলুন): আপনার ছয়টি ঘরের কোনটায় এখনো **একটিও সংখ্যা নেই**? ওই ঘরটাই আপনার প্রজেক্টে সবচেয়ে দেরিতে ফেটে পড়বে।

শিক্ষকের নোট: ২৫ মিনিট শেষে ২–৩ জনের পেপার বোর্ডে টাঙিয়ে একসাথে দেখুন। যার খালি ঘর সবচেয়ে বেশি, তার পেপারটাই সবচেয়ে বেশি শেখাবে — সেটাই বেছে নিন।

০৫

পর্ব ১ শেষ — পর্ব ২: পরিকল্পনা

ক্লাস ৫ থেকে: প্রয়োজন চিহ্নিত করা, ব্যবহারকারীর গল্প, আর গ্রহণযোগ্যতার শর্ত